NOUS AI ACADEMY / INTENSIVE FILE

TECHNICAL LEARNING FILE

DEEP-19

Reinforcement Learning

Sequential decisions and safe evaluation

FORMAT	LENGTH	ACCESS
INTENSIVE FILE	50 PAGES	OFFLINE

Edition 2 - original curriculum - editorial review recommended

FILE / ORIENTATION

Purpose

01 FILE GUIDANCE

This optional advanced guide does not gate the core learning path. Apply reinforcement learning with explicit assumptions, tests, tradeoffs, and limitations.

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.

FILE / ORIENTATION

Prerequisites

01 FILE GUIDANCE

Complete the related core track or pass its diagnostic.

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.

FILE / ORIENTATION

Study Method

01 FILE GUIDANCE

For each unit, explain the concept, follow the workflow, reproduce the example, analyze a failure, and complete the applied lab. Record evidence locally.

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.

UNIT 01 / CONCEPT

Agents and Environments: Concept

01 OBJECTIVE

Explain and apply agents and environments using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Agents and Environments is a central part of reinforcement learning because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Reinforcement Learning, the terms Agents, Environments are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should agents and environments be used, and what observable evidence would make that choice defensible? Build a small local example that applies agents and environments, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Agents and Environments in one precise sentence and name one assumption.
2. Create a two-step example of agents and environments and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 01 / WORKFLOW

Agents and Environments: Workflow

01 OBJECTIVE

Explain and apply agents and environments using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Agents and Environments is a central part of reinforcement learning because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to agents and environments.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies agents and environments, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Agents and Environments in one precise sentence and name one assumption.
2. Create a two-step example of agents and environments and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 01 / WORKED EXAMPLE

Agents and Environments: Worked Example

01 OBJECTIVE

Explain and apply agents and environments using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies agents and environments, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns agents and environments into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Agents and Environments in one precise sentence and name one assumption.
2. Create a two-step example of agents and environments and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 01 / FAILURE ANALYSIS

Agents and Environments: Failure Analysis

01 OBJECTIVE

Explain and apply agents and environments using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how agents and environments can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to agents and environments. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Agents and Environments in one precise sentence and name one assumption.
2. Create a two-step example of agents and environments and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 01 / APPLIED LAB

Agents and Environments: Applied Lab

01 OBJECTIVE

Explain and apply agents and environments using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of agents and environments into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies agents and environments, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Agents and Environments in one precise sentence and name one assumption.
2. Create a two-step example of agents and environments and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 02 / CONCEPT

Rewards: Concept

01 OBJECTIVE

Explain and apply rewards using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Rewards is a central part of reinforcement learning because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Reinforcement Learning, the terms Rewards are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should rewards be used, and what observable evidence would make that choice defensible? Build a small local example that applies rewards, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Rewards in one precise sentence and name one assumption.
2. Create a two-step example of rewards and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 02 / WORKFLOW

Rewards: Workflow

01 OBJECTIVE

Explain and apply rewards using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Rewards is a central part of reinforcement learning because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to rewards.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies rewards, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Rewards in one precise sentence and name one assumption.
2. Create a two-step example of rewards and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 02 / WORKED EXAMPLE

Rewards: Worked Example

01 OBJECTIVE

Explain and apply rewards using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies rewards, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns rewards into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Rewards in one precise sentence and name one assumption.
2. Create a two-step example of rewards and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 02 / FAILURE ANALYSIS

Rewards: Failure Analysis

01 OBJECTIVE

Explain and apply rewards using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how rewards can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to rewards. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Rewards in one precise sentence and name one assumption.
2. Create a two-step example of rewards and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 02 / APPLIED LAB

Rewards: Applied Lab

01 OBJECTIVE

Explain and apply rewards using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of rewards into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies rewards, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Rewards in one precise sentence and name one assumption.
2. Create a two-step example of rewards and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 03 / CONCEPT

Bandits: Concept

01 OBJECTIVE

Explain and apply bandits using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Bandits is a central part of reinforcement learning because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Reinforcement Learning, the terms Bandits are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should bandits be used, and what observable evidence would make that choice defensible? Build a small local example that applies bandits, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Bandits in one precise sentence and name one assumption.
2. Create a two-step example of bandits and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 03 / WORKFLOW

Bandits: Workflow

01 OBJECTIVE

Explain and apply bandits using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Bandits is a central part of reinforcement learning because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to bandits.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies bandits, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Bandits in one precise sentence and name one assumption.
2. Create a two-step example of bandits and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 03 / WORKED EXAMPLE

Bandits: Worked Example

01 OBJECTIVE

Explain and apply bandits using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies bandits, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns bandits into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Bandits in one precise sentence and name one assumption.
2. Create a two-step example of bandits and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 03 / FAILURE ANALYSIS

Bandits: Failure Analysis

01 OBJECTIVE

Explain and apply bandits using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how bandits can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to bandits. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Bandits in one precise sentence and name one assumption.
2. Create a two-step example of bandits and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 03 / APPLIED LAB

Bandits: Applied Lab

01 OBJECTIVE

Explain and apply bandits using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of bandits into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies bandits, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Bandits in one precise sentence and name one assumption.
2. Create a two-step example of bandits and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 04 / CONCEPT

Markov Decisions: Concept

01 OBJECTIVE

Explain and apply markov decisions using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Markov Decisions is a central part of reinforcement learning because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Reinforcement Learning, the terms Markov, Decisions are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should markov decisions be used, and what observable evidence would make that choice defensible? Build a small local example that applies markov decisions, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Markov Decisions in one precise sentence and name one assumption.
2. Create a two-step example of markov decisions and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 04 / WORKFLOW

Markov Decisions: Workflow

01 OBJECTIVE

Explain and apply markov decisions using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Markov Decisions is a central part of reinforcement learning because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to markov decisions.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies markov decisions, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Markov Decisions in one precise sentence and name one assumption.
2. Create a two-step example of markov decisions and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 04 / WORKED EXAMPLE

Markov Decisions: Worked Example

01 OBJECTIVE

Explain and apply markov decisions using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies markov decisions, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns markov decisions into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Markov Decisions in one precise sentence and name one assumption.
2. Create a two-step example of markov decisions and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 04 / FAILURE ANALYSIS

Markov Decisions: Failure Analysis

01 OBJECTIVE

Explain and apply markov decisions using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how markov decisions can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to markov decisions. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Markov Decisions in one precise sentence and name one assumption.
2. Create a two-step example of markov decisions and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 04 / APPLIED LAB

Markov Decisions: Applied Lab

01 OBJECTIVE

Explain and apply markov decisions using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of markov decisions into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies markov decisions, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Markov Decisions in one precise sentence and name one assumption.
2. Create a two-step example of markov decisions and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 05 / CONCEPT

Value Functions: Concept

01 OBJECTIVE

Explain and apply value functions using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Value Functions is a central part of reinforcement learning because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Reinforcement Learning, the terms Value, Functions are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should value functions be used, and what observable evidence would make that choice defensible? Build a small local example that applies value functions, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Value Functions in one precise sentence and name one assumption.
2. Create a two-step example of value functions and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 05 / WORKFLOW

Value Functions: Workflow

01 OBJECTIVE

Explain and apply value functions using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Value Functions is a central part of reinforcement learning because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to value functions.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies value functions, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Value Functions in one precise sentence and name one assumption.
2. Create a two-step example of value functions and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 05 / WORKED EXAMPLE

Value Functions: Worked Example

01 OBJECTIVE

Explain and apply value functions using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies value functions, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns value functions into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Value Functions in one precise sentence and name one assumption.
2. Create a two-step example of value functions and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 05 / FAILURE ANALYSIS

Value Functions: Failure Analysis

01 OBJECTIVE

Explain and apply value functions using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how value functions can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to value functions. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Value Functions in one precise sentence and name one assumption.
2. Create a two-step example of value functions and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 05 / APPLIED LAB

Value Functions: Applied Lab

01 OBJECTIVE

Explain and apply value functions using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of value functions into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies value functions, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Value Functions in one precise sentence and name one assumption.
2. Create a two-step example of value functions and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 06 / CONCEPT

Q-Learning: Concept

01 OBJECTIVE

Explain and apply q-learning using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Q-Learning is a central part of reinforcement learning because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Reinforcement Learning, the terms Learning are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should q-learning be used, and what observable evidence would make that choice defensible? Build a small local example that applies q-learning, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Q-Learning in one precise sentence and name one assumption.
2. Create a two-step example of q-learning and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 06 / WORKFLOW

Q-Learning: Workflow

01 OBJECTIVE

Explain and apply q-learning using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Q-Learning is a central part of reinforcement learning because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to q-learning.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies q-learning, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Q-Learning in one precise sentence and name one assumption.
2. Create a two-step example of q-learning and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 06 / WORKED EXAMPLE

Q-Learning: Worked Example

01 OBJECTIVE

Explain and apply q-learning using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies q-learning, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns q-learning into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Q-Learning in one precise sentence and name one assumption.
2. Create a two-step example of q-learning and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 06 / FAILURE ANALYSIS

Q-Learning: Failure Analysis

01 OBJECTIVE

Explain and apply q-learning using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how q-learning can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to q-learning. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Q-Learning in one precise sentence and name one assumption.
2. Create a two-step example of q-learning and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 06 / APPLIED LAB

Q-Learning: Applied Lab

01 OBJECTIVE

Explain and apply q-learning using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of q-learning into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies q-learning, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Q-Learning in one precise sentence and name one assumption.
2. Create a two-step example of q-learning and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 07 / CONCEPT

Policy Methods: Concept

01 OBJECTIVE

Explain and apply policy methods using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Policy Methods is a central part of reinforcement learning because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Reinforcement Learning, the terms Policy, Methods are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should policy methods be used, and what observable evidence would make that choice defensible? Build a small local example that applies policy methods, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Policy Methods in one precise sentence and name one assumption.
2. Create a two-step example of policy methods and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 07 / WORKFLOW

Policy Methods: Workflow

01 OBJECTIVE

Explain and apply policy methods using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Policy Methods is a central part of reinforcement learning because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to policy methods.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies policy methods, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Policy Methods in one precise sentence and name one assumption.
2. Create a two-step example of policy methods and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 07 / WORKED EXAMPLE

Policy Methods: Worked Example

01 OBJECTIVE

Explain and apply policy methods using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies policy methods, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns policy methods into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Policy Methods in one precise sentence and name one assumption.
2. Create a two-step example of policy methods and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 07 / FAILURE ANALYSIS

Policy Methods: Failure Analysis

01 OBJECTIVE

Explain and apply policy methods using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how policy methods can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to policy methods. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Policy Methods in one precise sentence and name one assumption.
2. Create a two-step example of policy methods and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 07 / APPLIED LAB

Policy Methods: Applied Lab

01 OBJECTIVE

Explain and apply policy methods using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of policy methods into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies policy methods, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Policy Methods in one precise sentence and name one assumption.
2. Create a two-step example of policy methods and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 08 / CONCEPT

Offline Evaluation: Concept

01 OBJECTIVE

Explain and apply offline evaluation using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Offline Evaluation is a central part of reinforcement learning because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Reinforcement Learning, the terms Offline, Evaluation are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should offline evaluation be used, and what observable evidence would make that choice defensible? Build a small local example that applies offline evaluation, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Offline Evaluation in one precise sentence and name one assumption.
2. Create a two-step example of offline evaluation and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 08 / WORKFLOW

Offline Evaluation: Workflow

01 OBJECTIVE

Explain and apply offline evaluation using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Offline Evaluation is a central part of reinforcement learning because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to offline evaluation.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies offline evaluation, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Offline Evaluation in one precise sentence and name one assumption.
2. Create a two-step example of offline evaluation and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 08 / WORKED EXAMPLE

Offline Evaluation: Worked Example

01 OBJECTIVE

Explain and apply offline evaluation using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies offline evaluation, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns offline evaluation into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Offline Evaluation in one precise sentence and name one assumption.
2. Create a two-step example of offline evaluation and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 08 / FAILURE ANALYSIS

Offline Evaluation: Failure Analysis

01 OBJECTIVE

Explain and apply offline evaluation using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how offline evaluation can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to offline evaluation. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Offline Evaluation in one precise sentence and name one assumption.
2. Create a two-step example of offline evaluation and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 08 / APPLIED LAB

Offline Evaluation: Applied Lab

01 OBJECTIVE

Explain and apply offline evaluation using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of offline evaluation into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies offline evaluation, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Offline Evaluation in one precise sentence and name one assumption.
2. Create a two-step example of offline evaluation and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 09 / CONCEPT

Reward Failure Modes: Concept

01 OBJECTIVE

Explain and apply reward failure modes using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Reward Failure Modes is a central part of reinforcement learning because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Reinforcement Learning, the terms Reward, Failure, Modes are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should reward failure modes be used, and what observable evidence would make that choice defensible? Build a small local example that applies reward failure modes, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Reward Failure Modes in one precise sentence and name one assumption.
2. Create a two-step example of reward failure modes and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 09 / WORKFLOW

Reward Failure Modes: Workflow

01 OBJECTIVE

Explain and apply reward failure modes using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Reward Failure Modes is a central part of reinforcement learning because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to reward failure modes.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies reward failure modes, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Reward Failure Modes in one precise sentence and name one assumption.
2. Create a two-step example of reward failure modes and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 09 / WORKED EXAMPLE

Reward Failure Modes: Worked Example

01 OBJECTIVE

Explain and apply reward failure modes using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies reward failure modes, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns reward failure modes into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Reward Failure Modes in one precise sentence and name one assumption.
2. Create a two-step example of reward failure modes and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 09 / FAILURE ANALYSIS

Reward Failure Modes: Failure Analysis

01 OBJECTIVE

Explain and apply reward failure modes using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how reward failure modes can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to reward failure modes. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Reward Failure Modes in one precise sentence and name one assumption.
2. Create a two-step example of reward failure modes and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 09 / APPLIED LAB

Reward Failure Modes: Applied Lab

01 OBJECTIVE

Explain and apply reward failure modes using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of reward failure modes into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies reward failure modes, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Reward Failure Modes in one precise sentence and name one assumption.
2. Create a two-step example of reward failure modes and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

FILE / ORIENTATION

Deep-Dive Capstone

01 FILE GUIDANCE

Build a compact, reproducible artifact demonstrating reinforcement learning. Include assumptions, a baseline, tests, failure analysis, results, limitations, and instructions for repeating the work offline.

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.